each element in the group. Inside the function, we call cy.wrap, which allows us to treat each individual element as a Cypress object for the purposes of chaining and asynchronous behavior, then we use should to test whether each element should be visible. In the first test, where no calendar filters have been clicked, everything is visible.

In the second test, we get the first calendar item and click on it, then test that the item has gained its border. Then we do the same visibility check, except that only the concerts associated with the day that was clicked should be visible. The others should, and I quote, not.be.visible.

The test for the Show All button uses a similar set of assertions but a different click pattern: We click on two of the calendars, then click the Show All button to verify the page changes back.

The search test uses the type action:

chapter_15/02/cypress/integration/schedule/schedule_spec.js

```
describe("search", () => {
  beforeEach(() => {
    cy.visit("/")
  })

  it("updates on search typing", function () {
    cy.get("#search_query").type("billy")
    cy.get("#search-results").as("searchResults")
    cy.get("@searchResults")
      .find("article")
      .find("article")
      .should("have.lengthOf", 1)
    cy.get("@searchResults").should("contain", "Billy")
  })
})
```

Here we're grabbing the #search_query element to find the box and type to put text in it. For the rest of it, we're checking that the contents of the search results contain the element that we expect.